AI Use Declaration Form

Course: Introduction to Artificial Intelligence
Lab Title: Lab 1 Part A — Uninformed Search Algorithms
Student Name: ___________Mariem Sall____
Student ID: _____________3043027____
GitHub Repository Link: https://github.com/Mariem-mcs/Mariem_Lab1
Date Submitted: ___20/06/2026____

1. AI Use Summary

Question	**Student Response**
Did you use any AI tool for this lab?	**Yes**
Estimated percentage of the work influenced by AI	% 12
Did you attach evidence of AI use where applicable?	**Yes**

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	**Purpose for Using the Tool**	**Prompt or Instruction Given to the Tool**	**Part(s) of the Work Influenced by the Tool**	**How I Verified, Edited, or Improved the AI Output**

Deepseek	**To debug my errors, and generate examples to test my codes**	**Why this error happened:**
		• `algorithm` is a variable that doesn't exist in that scope
		• Each `result` object has an `algorithm` attribute that you can access with
		• So you need `result.algorithm` not just `algorithm`
		
		Just replace the buggy code with:
		python
		`for result in results:`
		`    print(f"\n{result.algorithm}:")`
		`    print(f"  Status: {result.status}")`
		`    print(f"  Solution depth: {result.solution_depth}")`
			13. Run the Algorithms on the Sample Map	Before using the AI, I was getting errors when running the code I had, but after the use of AI code the output was what is asked.

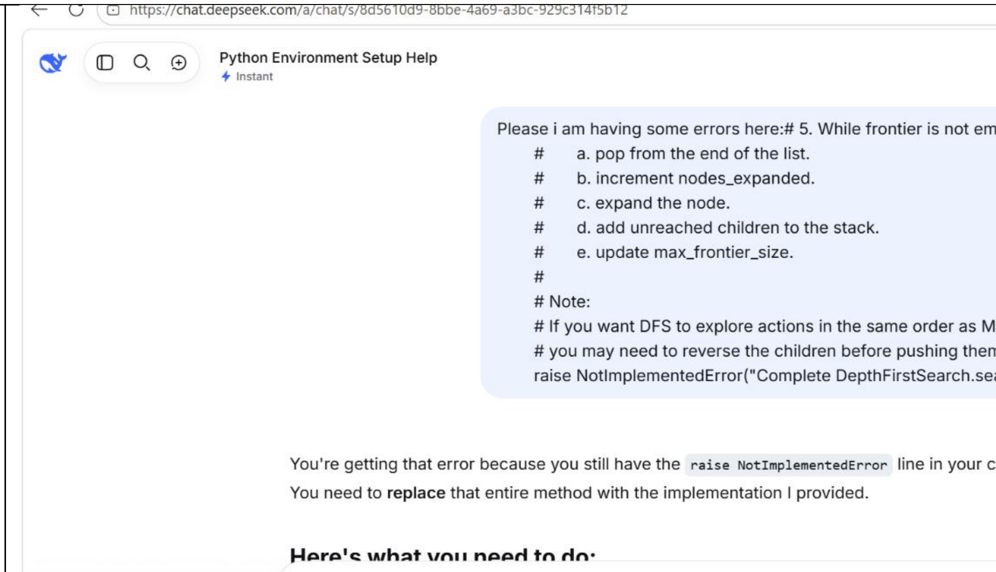
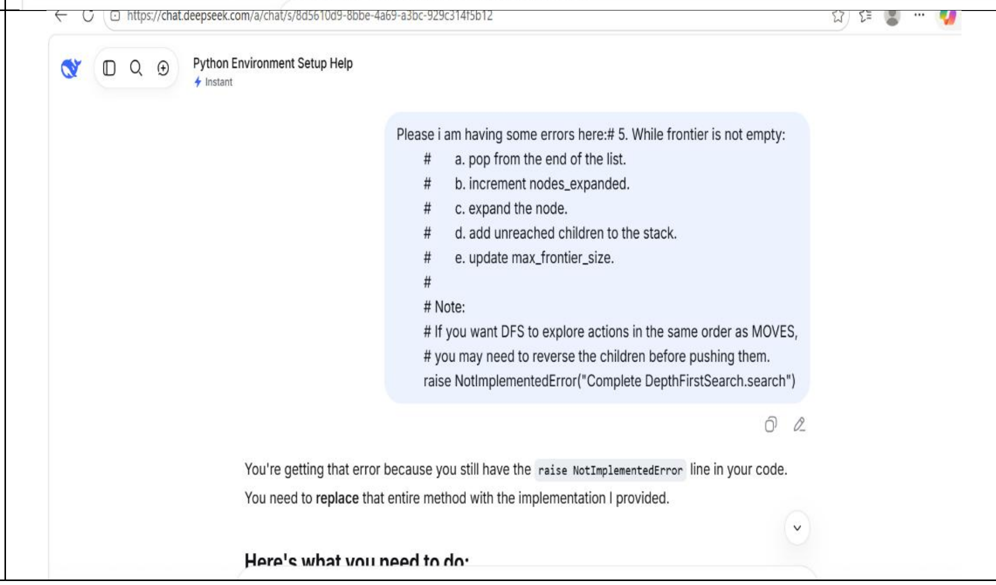
3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	**Attached? (Yes, No, N/A)**	**File Name**
AI-generated draft	from abc import ABC, abstractmethod from dataclasses import dataclass from collections import deque from typing import Any, Dict, Iterable, List, Optional, Tuple, Callable import numpy as np import pandas as pd import matplotlib.pyplot as plt import matplotlib.patches as patches import heapq import math MOVES = {	

```
"UP": (-1, 0),
"DOWN": (1, 0),
"LEFT": (0, -1),
"RIGHT": (0, 1),
}

def zero_heuristic(state, goal):
    return 0.0

def manhattan_distance(state, goal):
    r1, c1 = state
    r2, c2 = goal
    return abs(r1 - r2) + abs(c1 - c2)

def euclidean_distance(state, goal):
    r1, c1 = state
    r2, c2 = goal
    return math.sqrt((r1 - r2)**2 + (c1 - c2)**2)


class BestFirstSearch(SearchAlgorithm):
    def __init__(self, heuristic=zero_heuristic):
        self.heuristic = heuristic
        self.algorithm_name = "BestFirst"

    def h(self, node, problem):
        return self.heuristic(node.state, problem.goal)

    def evaluation(self, node, problem):
        raise NotImplementedError("Subclasses must define evaluation")

    def search(self, problem):
        initial_node = Node(problem.initial_state())

        # If start is goal
        if problem.is_goal(initial_node.state):
            return SearchResult(
                algorithm=self.algorithm_name,
                status="success",
                solution=initial_node,
                nodes_expanded=0,
                max_frontier_size=1,
                reached_count=1,
            )

        # Priority queue: (f_value, counter, node)
```

	counter = 0
	frontier = []
	initial_f = self.evaluation(initial_node, problem)
	heapq.heappush(frontier, (initial_f, counter, initial_node))
	counter += 1
	
	# Reached dictionary: state -> best node
	reached = {problem.initial_state(): initial_node}
	nodes_expanded = 0
	max_frontier_size = 1
	
	while frontier:
	_, _, node = heapq.heappop(frontier)
	
	# Late goal test
	if problem.is_goal(node.state):
	return SearchResult(
	algorithm=self.algorithm_name,
	status="success",
	solution=node,
	nodes_expanded=nodes_expanded,
	max_frontier_size=max_frontier_size,
	reached_count=len(reached),
	)
	
	nodes_expanded += 1
	
	for child in self.expand(problem, node):
	s = child.state
	if s not in reached or child.path_cost < reached[s].path_cost:
	reached[s] = child
	child_f = self.evaluation(child, problem)
	heapq.heappush(frontier, (child_f, counter, child))
	counter += 1
	max_frontier_size = max(max_frontier_size, len(frontier))
	
	return SearchResult(
	algorithm=self.algorithm_name,
	status="failure",
	solution=None,
	nodes_expanded=nodes_expanded,
	max_frontier_size=max_frontier_size,
	reached_count=len(reached),
	)	

Prompt history		
Screenshot of AI interaction		

the error i am getting now:---

NameError   Traceback (most recent call last)
Cell In[6], line 83
     79 print("DETAILED COMPARISON")
     80 print("=" * 70)
     81
     82 for result in results:
---> 83   print(f"\n{algorithm}:")
     84   print(f" Status: {result.status}")
     85   print(f" Solution depth: {result.solution_depth}")
     86   print(f" Solution cost: {result.solution_cost}")

NameError: name 'algorithm' is not defined

Python Environment Setup Help
Instant

Why Do We Use `_, _, node` Instead of `priority, counter, node`?

We **could** use:

```python
priority, counter, node = heapq.heappop(frontier)
```

But:

- We **don't need** `priority` (we already know it's the smallest)
- We **don't need** `counter` (it's just for breaking ties)
- We **only need** `node`

So using `_, _, node` is cleaner and tells other programmers:

> "I only care about the node, ignore the rest"

Exported AI conversation		
AI-generated code snippet		
Revised version showing student input		

4. Student Declaration

I declare that:

Declaration Statement	**Response (Yes/No)**
The submitted work is my own work.	Yes
Any use of AI tools has been clearly declared in this form.	Yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	Yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	Yes
My AI usage does not exceed 25% of the entire work.	Yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	Yes

Student Signature: _________________salll_____________________________

Date: _________________20/06/2026_____________________________